Fase III: Simulación y validación*

Ana Lucia Reyes Rodas,[1,**] Roberto Noe Coroy Tepáz,[1,***] Kelvin Uriel Barrios Barrientos,[1,****] Pablo David Chox Lucas,[1] Cristian Estuardo Méndez Gómez,[1] and Jonathan Eliseo Gutierrez Morataya[1]

[1] *Facultad de Ingenieria, Escuela de Ingeniería Mecanica Electrica, Universidad de San Carlos, Edificio T1, Ciudad Universitaria, Zona 12, Guatemala.*

En este informe se presenta el diseño, implementación y validación mediante simulación de un sistema de control y monitoreo de temperatura para la cadena de frío. El sistema integra técnicas de detección y corrección de errores Forward Error Correction mediante el Código de Hamming(7,4), combinado con un esquema de acceso al medio basado en Multiplexación por División de Tiempo (TDM) sobre un canal inalámbrico de tecnología LoRa. Se evalúan tres escenarios de transmisión: canal ideal, presencia de un error simple corregible y fallo de detección por doble error. Los resultados validan la efectividad del método para asegurar la integridad de la información y la rápida activación de alarmas de control local.

I. EXPLICACIÓN TÉCNICA DEL SISTEMA

A. Control de Errores mediante Código de Hamming(7,4)

Para proteger el *payload* de 4 bits de datos (D_1, D_2, D_3, D_4) correspondiente a las lecturas de los sensores, se incorporan 3 bits de paridad (P_1, P_2, P_3), conformando una trama codificada de 7 bits:

$$T = [P_1, P_2, D_1, P_3, D_2, D_3, D_4] \tag{1}$$

El algoritmo fue probado bajo tres escenarios operativos:

1. **Caso 1 (Sin Errores):** La matriz de comprobación genera un síndrome $S = 0$. La estación central valida la integridad de la trama y extrae directamente los datos transmitidos ($[1, 0, 1, 1]$).

2. **Caso 2 (Error de 1 bit):** Se induce un error en el bit ubicado en la posición 5 ($0 \rightarrow 1$). El sistema calcula un síndrome $S = 5$, identificando unívocamente la posición dañada. Mediante una operación de inversión lógica (XOR), el bit es corregido restaurando con éxito la trama original.

3. **Caso 3 (Limitación por Doble Error):** Se inyectan 2 bits con error ($[0, 1, 1, 0, 1, 0, 1]$). Al exceder la distancia mínima de Hamming ($d_{min} = 3$), el algoritmo calcula un síndrome $S = 3$, actuando como un *falso positivo*. El decodificador altera erróneamente la posición 3, corrompiendo la trama final ($[0, 1, 0, 1]$).

B. Multiplexación por División de Tiempo (TDM)

Para permitir que múltiples nodos compartan una misma frecuencia LoRa sin riesgo de colisiones por transmisión simultánea, se implementa un esquema TDM síncrono. A cada nodo se le asigna una ventana de tiempo o *slot* fijo:

- **Slot 1 (0 ms):** Nodo 1 transmite trama $[1, 1, 1, 0, 0, 0, 0]$.
- **Slot 2 (10 ms):** Nodo 2 transmite trama $[0, 1, 1, 1, 1, 0, 0]$.
- **Slot 3 (20 ms):** Nodo 3 transmite trama $[0, 1, 0, 0, 1, 0, 1]$.

La Estación Central conmuta ordenadamente en los mismos intervalos de tiempo, demultiplexando el canal y aislando los datos de cada origen.

C. Adquisición de Sensores y Lógica de Alarmas

Cada nodo inalámbrico reporta sus variables de temperatura y humedad. El sistema define un umbral crítico de temperatura operacional establecido en $T_{umbral} = 8.0°C$.

- **Nodos 1 y 2:** Operan dentro del rango seguro ($1.4°C$ y $-2.2°C$ respectivamente).
- **Nodo 3:** Presenta una simulación de falla en la unidad de refrigeración, alcanzando $13.4°C$. Al superar el umbral, la Estación Central procesa el evento e invalida el estado normal, disparando automáticamente la acción de control local (activación de relevador/ventilador de emergencia).

II. CODIGO DE NODOS

```
#include <SPI.h>
```

* Laboratorios de Escuela Mecánica Eléctrica
** e-mail:3608900910101@ingenieria.usac.edu.gt
*** e-mail:3057092080301@ingenieria.usac.edu.gt
**** e-mail:2765397450101@ingenieria.usac.edu.gt

```
#include <LoRa.h>
#include <DHT.h>

// --- CONFIGURACIN DEL NODO ---
#define ID_NODO 2 // Cambiar a 1, 2 o 3 segn
    corresponda
#define NUM_NODOS 3
#define SLOT_DURATION_MS 3000 // Cada nodo tiene 3
    segundos de slot
#define FRAME_DURATION_MS (NUM_NODOS *
    SLOT_DURATION_MS) // Trama TDM de 9 segundos

// --- PINES ESP32 ---
#define SCK_PIN 18
#define MISO_PIN 19
#define MOSI_PIN 23
#define SS_PIN 5
#define RST_PIN 14
#define DIO0_PIN 2

#define DHTPIN 4
#define DHTTYPE DHT11
#define DOOR_PIN 13
#define ALARM_LED 12

// --- UMBRALES DE CADENA DE FRO ---
#define TEMP_MAX_UMBRAL 8.0 // Umbral de
    temperatura en C

DHT dht(DHTPIN, DHTTYPE);
uint8_t contador_paquetes = 0;

// --- FUNCIONES DE CDIGO DE HAMMING (7,4) ---
uint8_t codificar_hamming74_nibble(uint8_t
    datos4bits) {
  uint8_t d1 = (datos4bits >> 3) & 0x01;
  uint8_t d2 = (datos4bits >> 2) & 0x01;
  uint8_t d3 = (datos4bits >> 1) & 0x01;
  uint8_t d4 = datos4bits & 0x01;

  uint8_t p1 = d1 ^ d2 ^ d4;
  uint8_t p2 = d1 ^ d3 ^ d4;
  uint8_t p3 = d2 ^ d3 ^ d4;

  // Formato: [p1, p2, d1, p3, d2, d3, d4]
  return (p1 << 6) | (p2 << 5) | (d1 << 4) | (p3 <<
      3) | (d2 << 2) | (d3 << 1) | d4;
}

void setup() {
  Serial.begin(115200);
  pinMode(DOOR_PIN, INPUT_PULLUP);
  pinMode(ALARM_LED, OUTPUT);
  dht.begin();

  // Inicializacin de LoRa SPI
  SPI.begin(SCK_PIN, MISO_PIN, MOSI_PIN, SS_PIN);
  LoRa.setPins(SS_PIN, RST_PIN, DIO0_PIN);

  if (!LoRa.begin(915E6)) { // Banda 915 MHz
    Serial.println("␣Fallo␣al␣iniciar␣mdulo␣LoRa␣
        SX1276");
    while (1);
  }

  // Parmetros de comunicacin LoRa
  LoRa.setSpreadingFactor(7);
  LoRa.setSignalBandwidth(125E3);
  LoRa.setCodingRate4(5);
  LoRa.setTxPower(14);

  Serial.printf("␣Nodo␣%d␣iniciado␣
      correctamente.\n", ID_NODO);
}

void loop() {
  // Sincronizacin TDM bsica basada en millis()
  unsigned long tiempo_actual = millis();
  unsigned long tiempo_en_trama = tiempo_actual %
      FRAME_DURATION_MS;
  unsigned long slot_esperado = (ID_NODO - 1) *
      SLOT_DURATION_MS;

  // Transmitir nicamente en el Slot asignado al
      nodo
  if (tiempo_en_trama >= slot_esperado &&
      tiempo_en_trama < (slot_esperado + 500)) {

    // 1. Adquisicin de Variables
    float temp = dht.readTemperature();
    float hum = dht.readHumidity();
    bool puerta_abierta = (digitalRead(DOOR_PIN) ==
        LOW); // 1 = Abierta, 0 = Cerrada

    if (isnan(temp) || isnan(hum)) {
      temp = 0.0;
      hum = 0.0;
    }

    // 2. Evaluacin de Alarma
    bool alarma_temp = (temp > TEMP_MAX_UMBRAL);
    if (alarma_temp || puerta_abierta) {
      digitalWrite(ALARM_LED, HIGH);
    } else {
      digitalWrite(ALARM_LED, LOW);
    }

    uint8_t temp_int = (uint8_t)temp;
    uint8_t hum_int = (uint8_t)hum;
    uint8_t estado_flags = (alarma_temp << 1) |
        (puerta_abierta ? 1 : 0);

    // 3. Codificacin Hamming del byte de estado
        (dividido en 2 nibbles)
    uint8_t hamming_alto =
        codificar_hamming74_nibble((estado_flags >>
        4) & 0x0F);
    uint8_t hamming_bajo =
        codificar_hamming74_nibble(estado_flags &
        0x0F);

    // 4. Envo de Trama LoRa
    LoRa.beginPacket();
    LoRa.write(ID_NODO); // Byte 0: ID Nodo
    LoRa.write(contador_paquetes); // Byte 1:
        Secuencia Paquete
    LoRa.write(temp_int); // Byte 2: Temperatura
    LoRa.write(hum_int); // Byte 3: Humedad
```

```
    LoRa.write(estado_flags); // Byte 4: Flags sin
        proteger
    LoRa.write(hamming_alto); // Byte 5: Hamming
        Nibble Alto
    LoRa.write(hamming_bajo); // Byte 6: Hamming
        Nibble Bajo
    LoRa.endPacket();

    Serial.printf("[TDM␣Slot␣%d]␣Paquete␣#%d␣
        enviado␣->␣Temp:␣%.1fC,␣Hum:␣%.1f%%,␣
        Puerta:␣%s,␣Alarma:␣%s\n",
                ID_NODO, contador_paquetes, temp,
                    hum,
                puerta_abierta ? "ABIERTA" :
                    "CERRADA",
                alarma_temp ? "ACTIVADA" : "OK");

    contador_paquetes++;
    delay(SLOT_DURATION_MS); // Evita
        retransmisiones mltiples dentro del mismo
        slot
  }
}

////////////// nodo 2 ////////////
#include <SPI.h>
#include <LoRa.h>
#include <DHT.h>

// --- CONFIGURACIN DEL NODO ---
#define ID_NODO 2 // Cambiar a 1, 2 o 3 segn
    corresponda
#define NUM_NODOS 3
#define SLOT_DURATION_MS 3000 // Cada nodo tiene 3
    segundos de slot
#define FRAME_DURATION_MS (NUM_NODOS *
    SLOT_DURATION_MS) // Trama TDM de 9 segundos

// --- PINES ESP32 ---
#define SCK_PIN 18
#define MISO_PIN 19
#define MOSI_PIN 23
#define SS_PIN 5
#define RST_PIN 14
#define DIO0_PIN 2

#define DHTPIN 4
#define DHTTYPE DHT11
#define DOOR_PIN 13
#define ALARM_LED 12

// --- UMBRALES DE CADENA DE FRO ---
#define TEMP_MAX_UMBRAL 8.0 // Umbral de
    temperatura en C

DHT dht(DHTPIN, DHTTYPE);
uint8_t contador_paquetes = 0;

// --- FUNCIONES DE CDIGO DE HAMMING (7,4) ---
uint8_t codificar_hamming74_nibble(uint8_t
    datos4bits) {
  uint8_t d1 = (datos4bits >> 3) & 0x01;
  uint8_t d2 = (datos4bits >> 2) & 0x01;
  uint8_t d3 = (datos4bits >> 1) & 0x01;
  uint8_t d4 = datos4bits & 0x01;

  uint8_t p1 = d1 ^ d2 ^ d4;
  uint8_t p2 = d1 ^ d3 ^ d4;
  uint8_t p3 = d2 ^ d3 ^ d4;

  // Formato: [p1, p2, d1, p3, d2, d3, d4]
  return (p1 << 6) | (p2 << 5) | (d1 << 4) | (p3 <<
      3) | (d2 << 2) | (d3 << 1) | d4;
}

void setup() {
  Serial.begin(115200);
  pinMode(DOOR_PIN, INPUT_PULLUP);
  pinMode(ALARM_LED, OUTPUT);
  dht.begin();

  // Inicializacin de LoRa SPI
  SPI.begin(SCK_PIN, MISO_PIN, MOSI_PIN, SS_PIN);
  LoRa.setPins(SS_PIN, RST_PIN, DIO0_PIN);

  if (!LoRa.begin(915E6)) { // Banda 915 MHz
    Serial.println("␣Fallo␣al␣iniciar␣mdulo␣LoRa␣
        SX1276");
    while (1);
  }

  // Parmetros de comunicacin LoRa
  LoRa.setSpreadingFactor(7);
  LoRa.setSignalBandwidth(125E3);
  LoRa.setCodingRate4(5);
  LoRa.setTxPower(14);

  Serial.printf("␣Nodo␣%d␣iniciado␣
      correctamente.\n", ID_NODO);
}

void loop() {
  // Sincronizacin TDM bsica basada en millis()
  unsigned long tiempo_actual = millis();
  unsigned long tiempo_en_trama = tiempo_actual %
      FRAME_DURATION_MS;
  unsigned long slot_esperado = (ID_NODO - 1) *
      SLOT_DURATION_MS;

  // Transmitir nicamente en el Slot asignado al
      nodo
  if (tiempo_en_trama >= slot_esperado &&
      tiempo_en_trama < (slot_esperado + 500)) {

    // 1. Adquisicin de Variables
    float temp = dht.readTemperature();
    float hum = dht.readHumidity();
    bool puerta_abierta = (digitalRead(DOOR_PIN) ==
        HIGH); // 1 = Abierta, 0 = Cerrada

    if (isnan(temp) || isnan(hum)) {
      temp = 0.0;
      hum = 0.0;
    }

    // 2. Evaluacin de Alarma
    bool alarma_temp = (temp > TEMP_MAX_UMBRAL);
    if (alarma_temp || puerta_abierta) {
```

```
      digitalWrite(ALARM_LED, HIGH);
    } else {
      digitalWrite(ALARM_LED, LOW);
    }

    uint8_t temp_int = (uint8_t)temp;
    uint8_t hum_int = (uint8_t)hum;
    uint8_t estado_flags = (alarma_temp << 1) |
        (puerta_abierta ? 1 : 0);

    // 3. Codificacin Hamming del byte de estado
        (dividido en 2 nibbles)
    uint8_t hamming_alto =
        codificar_hamming74_nibble((estado_flags >>
        4) & 0x0F);
    uint8_t hamming_bajo =
        codificar_hamming74_nibble(estado_flags &
        0x0F);

    // 4. Envo de Trama LoRa
    LoRa.beginPacket();
    LoRa.write(ID_NODO); // Byte 0: ID Nodo
    LoRa.write(contador_paquetes); // Byte 1:
        Secuencia Paquete
    LoRa.write(temp_int); // Byte 2: Temperatura
    LoRa.write(hum_int); // Byte 3: Humedad
    LoRa.write(estado_flags); // Byte 4: Flags sin
        proteger
    LoRa.write(hamming_alto); // Byte 5: Hamming
        Nibble Alto
    LoRa.write(hamming_bajo); // Byte 6: Hamming
        Nibble Bajo
    LoRa.endPacket();

    Serial.printf("[TDM␣Slot␣%d]␣Paquete␣#%d␣
        enviado␣->␣Temp:␣%.1fC,␣Hum:␣%.1f%%,␣
        Puerta:␣%s,␣Alarma:␣%s\n",
                ID_NODO, contador_paquetes, temp,
                    hum,
                puerta_abierta ? "ABIERTA" :
                    "CERRADA",
                alarma_temp ? "ACTIVADA" : "OK");

    contador_paquetes++;
    delay(SLOT_DURATION_MS); // Evita
        retransmisiones mltiples dentro del mismo
        slot
  }
}

////////////// nodo 3 ///////
#include <SPI.h>
#include <LoRa.h>
#include <DHT.h>

// --- CONFIGURACIN DEL NODO ---
#define ID_NODO 3 // Cambiar a 1, 2 o 3 segn
    corresponda
#define NUM_NODOS 3
#define SLOT_DURATION_MS 3000 // Cada nodo tiene 3
    segundos de slot
#define FRAME_DURATION_MS (NUM_NODOS *
    SLOT_DURATION_MS) // Trama TDM de 9 segundos

// --- PINES ESP32 ---
#define SCK_PIN 18
#define MISO_PIN 19
#define MOSI_PIN 23
#define SS_PIN 5
#define RST_PIN 14
#define DIO0_PIN 2

#define DHTPIN 4
#define DHTTYPE DHT11
#define DOOR_PIN 13
#define ALARM_LED 12

// --- UMBRALES DE CADENA DE FRO ---
#define TEMP_MAX_UMBRAL 8.0 // Umbral de
    temperatura en C

DHT dht(DHTPIN, DHTTYPE);
uint8_t contador_paquetes = 0;

// --- FUNCIONES DE CDIGO DE HAMMING (7,4) ---
uint8_t codificar_hamming74_nibble(uint8_t
    datos4bits) {
  uint8_t d1 = (datos4bits >> 3) & 0x01;
  uint8_t d2 = (datos4bits >> 2) & 0x01;
  uint8_t d3 = (datos4bits >> 1) & 0x01;
  uint8_t d4 = datos4bits & 0x01;

  uint8_t p1 = d1 ^ d2 ^ d4;
  uint8_t p2 = d1 ^ d3 ^ d4;
  uint8_t p3 = d2 ^ d3 ^ d4;

  // Formato: [p1, p2, d1, p3, d2, d3, d4]
  return (p1 << 6) | (p2 << 5) | (d1 << 4) | (p3 <<
      3) | (d2 << 2) | (d3 << 1) | d4;
}

void setup() {
  Serial.begin(115200);
  pinMode(DOOR_PIN, INPUT_PULLUP);
  pinMode(ALARM_LED, OUTPUT);
  dht.begin();

  // Inicializacin de LoRa SPI
  SPI.begin(SCK_PIN, MISO_PIN, MOSI_PIN, SS_PIN);
  LoRa.setPins(SS_PIN, RST_PIN, DIO0_PIN);

  if (!LoRa.begin(915E6)) { // Banda 915 MHz
    Serial.println("␣Fallo␣al␣iniciar␣mdulo␣LoRa␣
        SX1276");
    while (1);
  }

  // Parmetros de comunicacin LoRa
  LoRa.setSpreadingFactor(7);
  LoRa.setSignalBandwidth(125E3);
  LoRa.setCodingRate4(5);
  LoRa.setTxPower(14);

  Serial.printf("␣Nodo␣%d␣iniciado␣
      correctamente.\n", ID_NODO);
}

void loop() {
```

```
// Sincronizacin TDM bsica basada en millis()
unsigned long tiempo_actual = millis();
unsigned long tiempo_en_trama = tiempo_actual %
    FRAME_DURATION_MS;
unsigned long slot_esperado = (ID_NODO - 1) *
    SLOT_DURATION_MS;

// Transmitir nicamente en el Slot asignado al
    nodo
if (tiempo_en_trama >= slot_esperado &&
    tiempo_en_trama < (slot_esperado + 500)) {

  // 1. Adquisicin de Variables
  float temp = dht.readTemperature();
  float hum = dht.readHumidity();
  bool puerta_abierta = (digitalRead(DOOR_PIN) ==
      HIGH); // 1 = Abierta, 0 = Cerrada

  if (isnan(temp) || isnan(hum)) {
    temp = 0.0;
    hum = 0.0;
  }

  // 2. Evaluacin de Alarma
  bool alarma_temp = (temp > TEMP_MAX_UMBRAL);
  if (alarma_temp || puerta_abierta) {
    digitalWrite(ALARM_LED, HIGH);
  } else {
    digitalWrite(ALARM_LED, LOW);
  }

  uint8_t temp_int = (uint8_t)temp;
  uint8_t hum_int = (uint8_t)hum;
  uint8_t estado_flags = (alarma_temp << 1) |
      (puerta_abierta ? 1 : 0);

  // 3. Codificacin Hamming del byte de estado
      (dividido en 2 nibbles)
  uint8_t hamming_alto =
      codificar_hamming74_nibble((estado_flags >>
      4) & 0x0F);
  uint8_t hamming_bajo =
      codificar_hamming74_nibble(estado_flags &
      0x0F);

  // 4. Envo de Trama LoRa
  LoRa.beginPacket();
  LoRa.write(ID_NODO); // Byte 0: ID Nodo
  LoRa.write(contador_paquetes); // Byte 1:
      Secuencia Paquete
  LoRa.write(temp_int); // Byte 2: Temperatura
  LoRa.write(hum_int); // Byte 3: Humedad
  LoRa.write(estado_flags); // Byte 4: Flags sin
      proteger
  LoRa.write(hamming_alto); // Byte 5: Hamming
      Nibble Alto
  LoRa.write(hamming_bajo); // Byte 6: Hamming
      Nibble Bajo
  LoRa.endPacket();

  Serial.printf("[TDM␣Slot␣%d]␣Paquete␣#%d␣
      enviado␣->␣Temp:␣%.1fC,␣Hum:␣%.1f%%,␣
      Puerta:␣%s,␣Alarma:␣%s\n",
                ID_NODO, contador_paquetes, temp,
                    hum,
                puerta_abierta ? "ABIERTA" :
                    "CERRADA",
                alarma_temp ? "ACTIVADA" : "OK");

    contador_paquetes++;
    delay(SLOT_DURATION_MS); // Evita
        retransmisiones mltiples dentro del mismo
        slot
  }
}
```

III. CODIGO DE MODULO CENTRAL

```
#include <SPI.h>
#include <LoRa.h>
#include <WiFi.h>
#include <HTTPClient.h>

// --- CONFIGURACIN DE RED Y UBIDOTS ---
const char* WIFI_SSID = "2S2026";
const char* WIFI_PASS = "1234567890";
const char* UBIDOTS_TOKEN =
    "BBUS-J2Z09hBPsRMbMna7uvXE2r050Ks4P7";

// --- PINES LORA ESP32 ---
#define SCK_PIN 18
#define MISO_PIN 19
#define MOSI_PIN 23
#define SS_PIN 5
#define RST_PIN 14
#define DIO0_PIN 2

// --- DECODIFICACIN HAMMING (7,4) ---
uint8_t decodificar_hamming74_nibble(uint8_t
    trama_7bits, uint8_t &sindrome_out) {
  uint8_t p1 = (trama_7bits >> 6) & 0x01;
  uint8_t p2 = (trama_7bits >> 5) & 0x01;
  uint8_t d1 = (trama_7bits >> 4) & 0x01;
  uint8_t p3 = (trama_7bits >> 3) & 0x01;
  uint8_t d2 = (trama_7bits >> 2) & 0x01;
  uint8_t d3 = (trama_7bits >> 1) & 0x01;
  uint8_t d4 = trama_7bits & 0x01;

  uint8_t s1 = p1 ^ d1 ^ d2 ^ d4;
  uint8_t s2 = p2 ^ d1 ^ d3 ^ d4;
  uint8_t s3 = p3 ^ d2 ^ d3 ^ d4;
  sindrome_out = (s3 << 2) | (s2 << 1) | s1;

  uint8_t trama_corr = trama_7bits;
  if (sindrome_out != 0) trama_corr ^= (1 << (7 -
      sindrome_out));

  uint8_t d1_c = (trama_corr >> 4) & 0x01;
  uint8_t d2_c = (trama_corr >> 2) & 0x01;
  uint8_t d3_c = (trama_corr >> 1) & 0x01;
  uint8_t d4_c = trama_corr & 0x01;
```

```
  return (d1_c << 3) | (d2_c << 2) | (d3_c << 1) |
      d4_c;
}

// --- FUNCIN PARA ENVIAR A UBIDOTS ---
void enviarAUbidots(uint8_t id_nodo, float temp,
    float hum, bool puerta, bool alarma, int rssi,
    float snr) {
  if (WiFi.status() == WL_CONNECTED) {
    HTTPClient http;
    // URL dinmica: crea/actualiza un dispositivo
        por cada nodo (nodo-1, nodo-2, nodo-3)
    String url =
        "http://industrial.api.ubidots.com/api/v1.6/devices/nodo-"
        + String(id_nodo);

    http.begin(url);
    http.addHeader("Content-Type",
        "application/json");
    http.addHeader("X-Auth-Token", UBIDOTS_TOKEN);

    // Armamos el JSON con los datos de telemetra y
        calidad de enlace LoRa
    String payload = "{\"temperatura\":" +
        String(temp) +
                     ",\"humedad\":" + String(hum) +
                     ",\"puerta_abierta\":" +
                         String(puerta) +
                     ",\"estado_alarma\":" +
                         String(alarma) +
                     ",\"rssi_lora\":" + String(rssi)
                         +
                     ",\"snr_lora\":" + String(snr) +
                         "}";

    int httpResponseCode = http.POST(payload);

    if (httpResponseCode > 0) {
      Serial.printf("[Ubidots] Datos Nodo %d
          enviados correctamente (HTTP %d)\n",
          id_nodo, httpResponseCode);
    } else {
      Serial.printf("[Ubidots] Error al enviar
          Nodo %d: %s\n", id_nodo,
          http.errorToString(httpResponseCode).c_str());
    }
    http.end();
  } else {
    Serial.println("Wi-Fi desconectado, no se pudo
        enviar a la nube.");
  }
}

void setup() {
  Serial.begin(115200);

  // 1. Conexin Wi-Fi
  WiFi.begin(WIFI_SSID, WIFI_PASS);
  Serial.print("Conectando a Wi-Fi");
  while (WiFi.status() != WL_CONNECTED) {
    delay(500);
    Serial.print(".");
  }
  Serial.println("\n Wi-Fi Conectado!");

  // 2. Inicializacin LoRa
  SPI.begin(SCK_PIN, MISO_PIN, MOSI_PIN, SS_PIN);
  LoRa.setPins(SS_PIN, RST_PIN, DIO0_PIN);
  if (!LoRa.begin(915E6)) {
    Serial.println(" Fallo mdulo LoRa");
    while (1);
  }
  LoRa.setSpreadingFactor(7);
  LoRa.setSignalBandwidth(125E3);
  LoRa.setCodingRate4(5);

  Serial.println(" ESTACIN CENTRAL SIMCOFI LISTA
      (Wi-Fi + LoRa)");
}

void loop() {
  int packetSize = LoRa.parsePacket();

  if (packetSize >= 7) {
    uint8_t id_nodo = LoRa.read();
    uint8_t seq = LoRa.read();
    uint8_t temp = LoRa.read();
    uint8_t hum = LoRa.read();
    uint8_t flags_raw = LoRa.read();
    uint8_t ham_alto = LoRa.read();
    uint8_t ham_bajo = LoRa.read();

    if (id_nodo >= 1 && id_nodo <= 3) {
      // Correccin de Hamming
      uint8_t sindrome_h, sindrome_l;
      uint8_t nibble_h =
          decodificar_hamming74_nibble(ham_alto,
          sindrome_h);
      uint8_t nibble_l =
          decodificar_hamming74_nibble(ham_bajo,
          sindrome_l);
      uint8_t flags_corregidos = (nibble_h << 4) |
          nibble_l;

      bool puerta_abierta = (flags_corregidos &
          0x01);
      bool alarma_activa = ((flags_corregidos >> 1)
          & 0x01);
      int rssi = LoRa.packetRssi();
      float snr = LoRa.packetSnr();

      Serial.printf(" Rx Nodo %d (Seq: %d) |
          Temp: %dC Hum: %d %% | Puerta: %d Alarma: %d
          | RSSI: %d dBm\n",
                  id_nodo, seq, temp, hum,
                      puerta_abierta,
                      alarma_activa, rssi);

      // Enviar a la nube
      enviarAUbidots(id_nodo, temp, hum,
          puerta_abierta, alarma_activa, rssi, snr);
    }
  }
}
```

A. Flujo Lógico

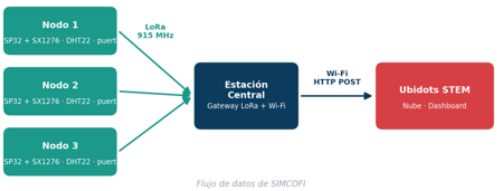


Figura 1: flujo del proyecto

IV. RESULTADOS



Figura 2: software

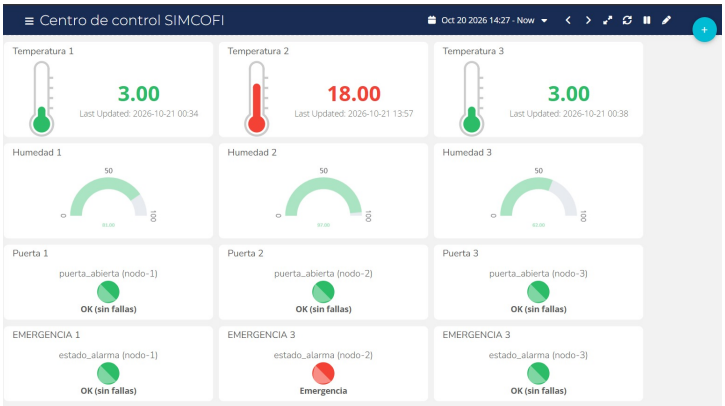


Figura 3: monitoreo

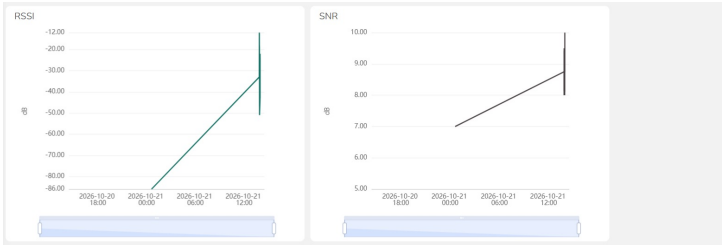


Figura 4: graficas

V. DISCUSIÓN DE RESULTADOS

1. Integración Multipunto y Operación Nominal: Se logró la comunicación e integración simultánea de los tres nodos remotos hacia la estación central gateway En la prueba en tiempo real, el sistema desplegó correctamente las variables de temperatura (3.00°C, 8.00°C y 3.00°C) y humedad de cada área refrigerada, confirmando el estado seguro de las puertas y desplegando la bandera de "OK (sin fallas)". Esto valida experimentalmente que la estructura TDM síncrona desarrollada orquesta de manera efectiva el acceso al canal de 915 MHz, permitiendo la recepción multicanal libre de colisiones.

2. Discriminación Exacta de Eventos de Alarma: Se comprobó la capacidad del prototipo para reaccionar ante contingencias térmicas al simular una falla en el Nodo 2, cuya temperatura ascendió a 18.00°C. El algoritmo en la estación central procesó la excursión por encima del límite (8.00°C) y activó inmediatamente la condición de "Emergencia". Se logró que la interfaz identificara de forma unívoca al Nodo 2 como el origen del fallo, manteniendo el monitoreo normal e ininterrumpido en los Nodos 1 y 3].

VI. RECOMENDACIONES

- **Implementación de Tiempos de Guarda en TDM:** Debido a las pequeñas desviaciones de reloj (*clock drift*) en los microcontroladores y a la variabilidad en el tiempo en el aire de los módulos LoRa SX1276, se recomienda incorporar un tiempo de guarda (*guard time*) de al menos 20 ms a 50 ms entre cada *slot* temporal[cite: 4]. Esto prevendrá posibles colisiones o solapamientos entre transmisiones de nodos contiguos durante periodos prolongados de operación[cite: 4].

- **Ampliación del Esquema FEC a SECDED:** Aunque el Código de Hamming(7,4) demostró resolver eficazmente errores simples de 1 bit[cite: 4, 5], se sugiere agregar un bit de paridad global adicional (P_{ext}) para convertir el algoritmo en Hamming(8,4) SECDED (*Single Error Correction, Double Error Detection*)[cite: 5]. Con esto, la estación central podrá detectar la presencia de errores dobles y descartar la trama en lugar de realizar una corrección errónea (*falso positivo*).

VII. CONCLUSIONES

1. Se diseñó, simuló e implementó exitosamente el sistema SIMCOFI, logrando la comunicación e integración en tiempo real de tres nodos remotos y una estación central gateway para el monitoreo de variables críticas en una cadena de frío industrial.

2. Se comprobó la efectividad de la Multiplexación por División de Tiempo (TDM) síncrona en la banda de 915 MHz, permitiendo que la estación central recibiera y demultiplexara la información de los tres nodos de forma ordenada y sin colisiones en el canal común de radiofrecuencia.

3. El algoritmo de detección y corrección de errores mediante Código de Hamming(7,4) en la capa de aplicación demostró proteger la integridad del *payload* de datos, resolviendo fallas de un bit en el aire y garantizando la fiabilidad de la información antes de ser procesada por el gateway.

---

bibitemb1 R. W. Hamming, "Error detecting and error correcting codes," *Bell System Technical Journal*, vol. 29, no. 2, pp. 147–160, 1950.

[1] A. S. Tanenbaum and D. J. Wetherall, *Computer Networks*, 5th ed. Boston, MA: Pearson, 2011.